

AmanaRAG MVP – Technical Specification

7-Day Development Sprint (Nov 22-28, 2024)

Project Description

AmanaRag is a BYOK (Bring Your Own Key) RAG-based chatbot platform that enables small businesses to embed an AI assistant on their website. The platform allows business owners to upload their own documents (PDFs, Q&A; pairs) and connect their OpenAI API key. The system handles document ingestion, embedding generation, vector storage, and intelligent retrieval to answer customer questions.

The MVP focuses on a single-site, single-admin setup with essential features: document upload, RAG pipeline, chat widget, and basic logging dashboard. This allows us to validate the core concept before scaling to multi-tenant architecture.

Key Innovation: Unlike competitors (Intercom, Crisp, Tidio), we charge only for platform usage while customers pay LLM providers directly using their own API keys – making it extremely affordable for small businesses.

Requirements

1. *Next.js Application – Pages*

- **Login / Registration Page** – Admin authentication using Supabase Auth.
- **Dashboard Home** – Overview with quick stats and site management.
- **Content Upload Page** – Upload PDFs and add Q&A; pairs manually.
- **API Key Setup Page** – Secure input for OpenAI API key with validation.
- **Chat Logs Page** – View all conversations between visitors and chatbot.
- **Widget Setup Page** – Generate and display embed script snippet.
- **Hosted Chat Page** – Public-facing chat interface loaded by the widget.

2. *Responsiveness*

- All dashboard pages will be fully responsive using Tailwind CSS.
- Chat widget adapts to mobile and desktop views.
- Purple (#6B46C1) theme with clean, modern UI components.

3. *RAG Pipeline Integration*

- Document chunking using LangChain text splitters.

- Embedding generation using OpenAI API (user's key).
- Vector storage in Supabase with pgvector extension.
- Top-k retrieval for context building.
- LLM completion using user's BYOK API key.

4. Deployment

- Frontend deployed on **Vercel** (Next.js native platform).
- Database and storage on **Supabase** (PostgreSQL + pgvector).
- Widget.js served via Vercel Edge CDN for fast global loading.

5. Form Validation & Security

- API keys encrypted at rest using environment variables.
- HTTPS enforced for all endpoints.
- Rate limiting on chat API to prevent abuse.
- Input sanitization to prevent injection attacks.
- Supabase Row Level Security (RLS) for data isolation.

6. Documentation & Repository

- Organized GitHub repo with clear folder structure.
- README.md with setup instructions, environment variables, and API docs.
- This PDF documentation for project overview.
- Clear commit history with meaningful messages.

7. Database (Supabase PostgreSQL + pgvector)

Our database uses 6 core tables to support the MVP:

Table	Description	Key Fields
users	Admin authentication	id, email, created_at
sites	Website configurations	id, user_id, name, api_key_encrypted, widget_settings
documents	Uploaded PDFs and files	id, site_id, filename, storage_path, status
qa_pairs	Manual Q&A; entries	id, site_id, question, answer
vectors	Embedded document chunks	id, site_id, doc_id, chunk_text, embedding, metadata
conversations	Chat logs	id, site_id, visitor_id, messages[], created_at

Note: pgvector extension must be enabled in Supabase for vector similarity search.

8. API Endpoints

Endpoint	Method	Purpose
/api/auth/signup	POST	Create admin account
/api/auth/login	POST	Authenticate admin
/api/content/upload	POST	Upload PDF document
/api/content/qa	POST	Add manual Q&A pair
/api/chat/:siteId	POST	Handle visitor chat messages (RAG)
/api/logs/:siteId	GET	Retrieve conversation logs
/api/widget/:siteId	GET	Get widget configuration
/api/site/key	POST	Save/update OpenAI API key

9. User Flows

Admin Flow (Business Owner):

1. Register/Login to dashboard
2. Set up site and enter OpenAI API key
3. Upload PDFs or add Q&A; pairs
4. System processes documents (chunking, embedding, storage)
5. Copy widget script from dashboard
6. Paste script into website HTML
7. Monitor chat logs and improve knowledge base

Visitor Flow (Website Customer):

1. Visit client website (e.g., x1.com)
2. Widget loads in bottom-right corner
3. Click to open chat interface
4. Ask question about the business
5. RAG system retrieves relevant docs
6. OpenAI generates answer using business context
7. Visitor receives accurate, helpful response
8. Conversation logged for admin review

10. Wireframes

[Wireframes to be added here - placeholder for:]

- Dashboard home screen
- Content upload page
- Chat logs view
- Widget preview and script generator
- Hosted chat interface (visitor view)

Design tool: Figma or Excalidraw

Color scheme: Purple (#6B46C1), White (#FFFFFF), Gray shades (#111827, #1F2937)

11. Tools & Technologies

Category	Tool/Technology
Frontend Framework	Next.js 13+ (App Router)
Styling	Tailwind CSS
Backend	Next.js API Routes
Database	Supabase (PostgreSQL + pgvector)

Authentication	Supabase Auth
File Storage	Supabase Storage
Vector Search	pgvector extension
RAG Framework	LangChain
LLM Provider	OpenAI (user's API key)
Deployment	Vercel + Supabase
Version Control	GitHub
Project Management	Linear or Notion

12. 7-Day Development Sprint

Day	Focus	Deliverables
Day 1 (Nov 22)	Setup & Auth	● Next.js project initialized ● Supabase configured ● Auth pages working ● Dashboard skeleton
Day 2 (Nov 23)	Content Upload	● PDF upload UI ● Q&A form ● File storage integration ● Basic validation
Day 3 (Nov 24)	RAG Pipeline	● Document chunking ● Embedding generation ● Vector storage (pgvector) ● API key management
Day 4 (Nov 25)	Chat Widget	● Widget.js script ● Hosted chat page ● Chat API endpoint ● Basic RAG retrieval
Day 5 (Nov 26)	Integration	● Connect all components ● RAG testing & tuning ● Error handling ● Logs page
Day 6 (Nov 27)	Polish	● UI refinements ● Bug fixes ● Performance optimization ● Documentation
Day 7 (Nov 28)	Deploy	● Production deployment ● Final testing ● Demo preparation ● Handoff docs

13. Team Responsibilities

Mahmoud (Backend + RAG Lead):

- RAG pipeline architecture
- Document chunking logic
- Embedding generation setup
- Vector database design
- Retrieval and ranking implementation
- API key encryption and validation
- Chat API endpoint logic

Aya (Frontend + Full-Stack Support):

- Next.js dashboard development

- All UI/UX pages (upload, logs, settings)
- Chat widget implementation (based on Amana-Chat repo)
- Tailwind component library
- API route scaffolding and integration
- Supabase client setup
- Responsive design for all pages
- Support Mahmoud with backend debugging

Notes for Aya

Some helpful context to keep in mind as you work on the frontend and backend integration:

- Mahmoud will be handling all RAG logic and implementation, so you can focus on the Next.js dashboard and API scaffolding.
- If you're still learning Next.js, that's perfectly fine – we'll keep the plan simple and linear to make it manageable.
- Your main focus will be on the backend (Node.js/Next.js API routes) and the simple frontend dashboard. The RAG complexity is handled separately.
- We're keeping the architecture straightforward to ensure we can deliver a clean MVP within the 7-day timeline.

14. MVP Limitations (v1)

- Single admin, single site only (no multi-tenant)
- No Stripe billing integration
- No advanced analytics dashboard
- No widget customization (theme is fixed purple)
- No multi-language support
- No human handoff or escalation
- No team collaboration features
- Basic logging only (no sentiment analysis)
- No A/B testing or experimentation tools

15. Future Roadmap (Post-MVP)

- **Phase 2:** Multi-site support per admin, improved analytics
- **Phase 3:** Multi-tenant architecture, Stripe integration, team roles
- **Phase 4:** Widget theming, white-label options, marketplace

- **Phase 5:** Mobile app, advanced RAG (re-ranking, hybrid search)
- **Phase 6:** AI-powered answer correction, sentiment analysis, A/B testing

16. Important Links

GitHub Repository:

[To be added - organized repo with clear README]

Project Management:

[Linear/Notion board link - track tasks and progress]

Deployment:

- Production: [Vercel app URL]
- Database: [Supabase project dashboard]

Reference Repo (Amana-Chat):

<https://github.com/AyaMehjez/Amana-Chat>
(Chat UI styling and component inspiration)

This documentation serves as the technical blueprint for the AmanaRag MVP. All team members should refer to this document for scope, responsibilities, and timelines. Update this doc as the project evolves.